

AI GATEWAY COMPARISON

Capabilities · Pros & Cons · Best Practices · Anti-Patterns

An Enterprise Architect's Reference covering Kong AI · LiteLLM · AWS Bedrock ·
Azure APIM · Portkey · Cloudflare · Apigee · WSO2 · Traefik · IBM watsonx

Enterprise AI Architecture Practice · April 2026 · Addendum 03
10 Tools · Version 1.0 · CONFIDENTIAL

MASTER CAPABILITY COMPARISON MATRIX

The matrix below scores each gateway across 10 capability dimensions on a scale of 1–10. Scores reflect the out-of-box capability without significant customisation. Green = strong (8–10), Yellow = moderate (5–7), Red = weak (1–4).

Tool	Provider Routing	Extensibility	Sem. Cache	Security & Guards	Observability	Ease of Setup	Multi-tenant Isolation	Multi-Cloud	OSS Maturity	FinOps
Kong AI	9/10	10/10	9/10	8/10	8/10	6/10	8/10	8/10	9/10	8/10
LiteLLM Proxy	10/10	6/10	8/10	6/10	7/10	9/10	6/10	9/10	8/10	9/10
AWS Bedrock	5/10	4/10	3/10	9/10	7/10	8/10	7/10	2/10	1/10	7/10
Azure API	5/10	6/10	7/10	7/10	7/10	7/10	7/10	3/10	2/10	7/10
Portkey.ai	9/10	6/10	7/10	6/10	9/10	9/10	6/10	8/10	6/10	8/10
Cloudflare AI	6/10	5/10	6/10	8/10	7/10	10/10	5/10	6/10	1/10	6/10
Google Apigee	5/10	7/10	8/10	7/10	8/10	6/10	8/10	3/10	1/10	7/10
WSO2 Choreo	7/10	7/10	6/10	6/10	7/10	5/10	7/10	7/10	8/10	6/10
Traefik AI	6/10	7/10	3/10	5/10	7/10	8/10	5/10	6/10	8/10	4/10
IBM watsonx	4/10	5/10	4/10	9/10	8/10	3/10	8/10	4/10	2/10	6/10

Legend: ■ 8–10 Strong ■ 5–7 Moderate ■ 1–4 Weak Scores reflect out-of-box capability without significant customisation.

01

Kong AI Gateway

The enterprise plugin-powered AI traffic controller

Kong Inc. (2024) · Apache 2 (OSS) / Enterprise · Enterprise / Cloud-native

Kong AI Gateway extends the battle-tested Kong Gateway with a dedicated AI plugin suite. It sits in front of any LLM provider, enforcing rate limits, semantic caching, PII redaction, and prompt guards through a declarative plugin chain. The plugin ecosystem (Lua, WASM, Go) makes it the most extensible option for platform teams already running Kong for API management, enabling a single pane of glass across REST, GraphQL, gRPC, and AI traffic.

Core Capabilities

Multi-provider Routing	Semantic Cache	Rate Limiting	Prompt Guard	PII Redaction	Plugin Ecosystem	mTLS	OpenTelemetry
RBAC / Workspaces	Kubernetes CRDs	AI Cost Tracking	Fallback Chains	Streaming SSE	Declarative Config (deck)		

Capability Scores

Multi-provider routing		9/10
Extensibility / Plugins		10/10
Semantic caching		9/10
Security & guardrails		8/10
Observability		8/10
Ease of setup		6/10
Multi-tenant isolation		8/10
Multi-cloud support		8/10
OSS maturity		9/10
FinOps		8/10

Pros / Cons / Best Practices / Anti-Patterns

■ PROS	✗ CONS
• Best-in-class plugin ecosystem — Lua, WASM, Go plugins for unlimited extensibility • Native Kubernetes Ingress Controller (KIC) and Gateway Operator (KGO) for cloud-native ops • Declarative config via deck CLI + GitOps — full IaC lifecycle • Semantic cache plugin with pgvector/Redis reduces token costs by 40–70% • Multi-workspace tenancy model for BU-level isolation out of the box • Strong OSS community + enterprise support with SLA guarantees • AI-specific plugins: ai-proxy, ai-rate-limiting, ai-semantic-cache, ai-prompt-guard • Provider abstraction — single route serves OpenAI, Azure, Bedrock, Claude	• Steeper learning curve than hosted alternatives — requires Kong expertise • Enterprise features (RBAC, Vault, advanced analytics) behind paid license • Plugin development requires Lua or WASM knowledge — not Python-native • Control plane (Kong Manager / Konnect) adds operational overhead • Semantic cache vector DB must be provisioned separately (pgvector, Redis Stack) • No native eval/quality scoring — requires external tooling
★ BEST PRACTICES	■ ANTI-PATTERNS
→ Deploy Kong Manager on a dedicated control plane node separate from data plane → Use deck sync in CI/CD to validate and apply config changes — never manual Admin API in prod → Namespace all plugin configs under Workspaces matching tenant hierarchy → Enable ai-semantic-cache with similarity threshold 0.85–0.90 to balance hit rate vs freshness → Use Vault KV secrets engine for all provider API keys — never hardcode in deck files → Set ai-rate-limiting-advanced with sliding_window strategy and per-consumer JWT sub claims → Deploy Kong with at least 3 replicas behind an NLB for HA; use PodDisruptionBudgets → Monitor gateway_ai_llm_usage_tokens metric to drive FinOps chargeback	! Running Kong in DB-less mode without a backup config store in production — single config loss point ! Installing too many plugins in the global scope — degrades all routes, not just AI routes ! Using API key auth without Vault rotation — keys in plaintext in deck YAML committed to git ! Setting rate limits by IP instead of JWT consumer — bypassed by shared NAT environments ! Ignoring Kong's upstream health checks — circuit breaker won't trigger without active probes ! Using the Community Edition for enterprise multi-tenancy — Workspaces require Enterprise ! Skipping load testing the semantic cache similarity threshold — too low = poor hit rate; too high = stale answers

Reference Snippet

■ Kong AI Gateway – Full AI route with guard + cache (snippet)

```
services:  
- name: openai-service  
  url: https://api.openai.com
```

```
routes:
- name: chat-route
paths: [/ai/v1]
service: openai-service
plugins:
- name: openid-connect # AuthN
config: { issuer: https://idp.corp/oidc }
- name: ai-proxy
config:
route_type: llm/v1/chat
model: { provider: openai, name: gpt-4o }
auth: { header_value: Bearer {vault://gw/openai-key} }
- name: ai-prompt-guard # Inject shield
config:
rules: [{ match: regex, pattern: 'ignore (all )?instructions', action: block }]
- name: ai-semantic-cache
config:
vectordb: { strategy: redis, threshold: 0.87 }
- name: ai-rate-limiting-advanced
config:
limit: [{ minute: 200, day: 20000 }]
identifier: consumer
```

VERDICT

Best choice for platform teams who already operate Kong or need maximum extensibility. The plugin model is unmatched. Requires investment in Kong expertise and enterprise license for full multi-tenancy.

02

LiteLLM Proxy

The open-source 100+ provider unification layer

BerriAI (2023) · MIT (OSS) / Enterprise · Dev teams / Platform engineering

LiteLLM Proxy is the most popular open-source AI gateway, providing a unified OpenAI-compatible endpoint in front of 100+ LLM providers. Written in Python, it is the natural choice for Python-native ML/AI teams. It ships with built-in semantic caching, budget limits, virtual keys, load balancing, and a management UI. The proxy is lightweight enough to run as a sidecar and powerful enough to serve as a production gateway for mid-scale deployments.

Core Capabilities

100+ Provider Support	Virtual API Keys	Budget Limits	Semantic Cache (Redis)	Load Balancing	Fallback Chains	Retry Logic	Spend Tracking
Prometheus Metrics	Callback Hooks	OpenAI-compatible API	Team Management UI	Model Aliases	Async Batch Processing		

Capability Scores

Multi-provider routing		10/10
Extensibility / Plugins		6/10
Semantic caching		8/10
Security & guardrails		6/10
Observability		7/10
Ease of setup		9/10
Multi-tenant isolation		6/10
Multi-cloud support		9/10
OSS maturity		8/10
FinOps		9/10

Pros / Cons / Best Practices / Anti-Patterns

■ PROS

- Supports 100+ providers out of the box — largest provider coverage in any gateway
- Python-native — callbacks, custom routers, and hooks written in native Python
- Virtual keys with per-key budget limits, model restrictions, and team assignment
- Built-in spend tracking and cost analytics dashboard without external tooling
- Latency-based routing and lowest-cost-routing strategies built in
- Seamless drop-in replacement — change `OPENAI_BASE_URL`, zero app code changes
- Active open-source community with weekly releases
- Lightest operational footprint — runs as a single Docker container

✗ CONS

- Isolation model is logical, not physical — all tenants share the same process
- Plugin extensibility limited to Python callbacks — no declarative plugin chain
- No native Kubernetes CRD / Operator — requires manual Helm chart management
- Prompt injection/guard features are callback-based, not inline pipeline plugins
- Performance ceiling lower than Nginx/Kong-based gateways at very high RPS (>10k/s)
- Multi-region deployment requires external orchestration — no native federation
- Enterprise support is commercial but less mature than Kong's offering

★ BEST PRACTICES

- Store all provider keys in environment variables or Vault — never in `config.yaml`
- Use virtual keys per team with explicit model allow-lists and `daily_budget_usd` limits
- Enable Redis semantic cache with a similarity threshold of 0.85 for best cost vs quality
- Deploy behind a Kubernetes Deployment with 3+ replicas and readiness probes on `/health`
- Use `router_settings.routing_strategy: latency-based-routing` for production SLA compliance
- Implement custom callbacks for PII scanning (Presidio integration) before provider calls
- Export Prometheus metrics to Grafana; alert on `litellm_requests_metric` drop rate > 1%
- Use `model_group_alias` to abstract provider details from application teams

■ ANTI-PATTERNS

- ! Running LiteLLM as a single replica without Redis — all budget state is in-memory and lost on restart
- ! Using the master key for application teams — create per-team virtual keys with scoped permissions
- ! Setting `fallback_models` without health checking — fallback to a broken provider adds latency, not resilience
- ! Ignoring `max_budget_duration` — without TTL, budgets accumulate and never reset
- ! Deploying without a reverse proxy (Nginx/Traefik) in front — LiteLLM is not hardened for direct internet exposure
- ! Using `success_callbacks` for audit logging without async offload — blocks the request path
- ! Assuming OpenAI-compatible means identical — some providers return subtly different error codes that callbacks must handle

Reference Snippet

■ LiteLLM Proxy – `config.yaml` with fallback + budget + cache (snippet)

```
model_list:  
- model_name: gpt-4o  
litellm_params:
```

```
model: openai/gpt-4o
api_key: os.environ/OPENAI_KEY
- model_name: gpt-4o # Azure fallback
litellm_params:
model: azure/gpt-4o
api_base: os.environ/AZURE_ENDPOINT
- model_name: gpt-4o # Bedrock fallback
litellm_params:
model: bedrock/anthropic.claude-3-5-sonnet-20241022-v2:0
router_settings:
routing_strategy: latency-based-routing
allowed_fails: 2
cooldown_time: 30
litellm_settings:
cache: true
cache_params: { type: redis, similarity_threshold: 0.85 }
success_callbacks: [prometheus, langfuse]
general_settings:
master_key: os.environ/MASTER_KEY
database_url: os.environ/POSTGRES_URL
```

VERDICT

Best choice for Python-native ML/AI teams who need fast deployment and broad provider coverage. Ideal for startups and mid-market. At enterprise scale (multi-region, hard isolation), augment with infrastructure-level controls.

03

AWS Bedrock Gateway

Native AWS multi-model gateway with Guardrails

Amazon Web Services (2023) · Proprietary (AWS managed) · AWS-centric enterprise

AWS Bedrock is Amazon's fully managed foundation model service that also functions as an AI gateway when combined with Bedrock Guardrails, IAM policies, VPC endpoints, and AWS API Gateway. It provides access to models from Anthropic, Meta, Mistral, Cohere, Amazon Titan, and Stability AI through a single AWS SDK call. Guardrails add content filtering, PII detection, grounding checks, and topic-based blocking. The integration with AWS IAM, CloudWatch, and PrivateLink makes it the natural fit for organisations deeply invested in the AWS ecosystem.

Core Capabilities

Multi-model Catalog (Anthropic/Meta/Mistral/Amazon)	Bedrock Guardrails	IAM-based AuthN/AuthZ	VPC PrivateLink	CloudWatch Metrics	CloudTrail Audit	Knowledge Bases (RAG)	Agents for Bedrock
Model Evaluation	Provisioned Throughput	Cross-region Inference	PrivateLink Endpoints	Data Encryption (KMS)	Batch Inference		

Capability Scores

Multi-provider routing		5/10
Extensibility / Plugins		4/10
Semantic caching		3/10
Security & guardrails		9/10
Observability		7/10
Ease of setup		8/10
Multi-tenant isolation		7/10
Multi-cloud support		2/10
OSS maturity		1/10
FinOps		7/10

Pros / Cons / Best Practices / Anti-Patterns

■ PROS	✗ CONS
• Zero infrastructure to manage — fully serverless, scales to zero • Native IAM integration — granular model access via IAM policies and SCPs • Bedrock Guardrails: content filters, PII redaction, grounding checks, topic blocks • VPC PrivateLink ensures prompts never traverse the public internet • Provisioned Throughput for guaranteed TPS with no rate-limit surprises • Knowledge Bases provides managed RAG with automatic embedding + vector store • Agents for Bedrock orchestrates multi-step tool-use workflows natively • CloudTrail provides immutable audit logs of every model invocation	• Hard AWS lock-in — no cross-cloud portability of guardrails or routing logic • Model selection limited to Bedrock catalog — no OpenAI GPT-4o, no on-prem models • No semantic caching — every request hits the model and incurs cost • Guardrails are coarse-grained compared to Kong's plugin granularity • No concept of virtual keys or per-team budget enforcement at the gateway layer • Cross-region inference latency can be unpredictable during failover • Custom model fine-tuning and BYOM (Bring Your Own Model) workflow is complex • API surface diverges from OpenAI schema — requires adapter code in applications
★ BEST PRACTICES	■ ANTI-PATTERNS
→ Use IAM condition keys (bedrock:InvokedModelId) for per-model access control per team → Enable Bedrock Guardrails on every production inference profile — not just sensitive routes → Use Provisioned Throughput for latency-critical workloads to avoid throttling at peak load → Route via VPC PrivateLink + Interface Endpoint — never via public Bedrock endpoint for prod data → Tag all Bedrock resources with CostCenter and TeamID for AWS Cost Explorer chargeback → Use Cross-Region Inference profiles for disaster recovery — test failover quarterly → Store KMS keys in AWS Secrets Manager, not in application config, for model-level encryption → Enable CloudTrail data events for Bedrock — default trail only captures management events	! Using AdministratorAccess IAM role for Bedrock calls — violates least privilege ! Relying on Bedrock Guardrails alone for security — it does not cover prompt injection comprehensively ! Not setting on-demand concurrency limits — runaway agents can exhaust account-level TPS quotas ! Mixing multi-tenant data in the same Knowledge Base without namespace isolation — cross-tenant leakage ! Hardcoding model IDs (anthropic.claude-3...) in application code — breaks on model deprecation ! Ignoring Cross-Region inference latency SLAs — failover regions may be 200ms+ slower ! Not using Provisioned Throughput for production — on-demand throttling causes unpredictable P99 spikes

Reference Snippet

■ AWS Bedrock – Guardrails + IAM policy (snippet)

```
# Guardrail creation (AWS CLI)
aws bedrock create-guardrail \
```

```
--name prod-guardrail \  
--content-policy-config 'filtersConfig=[{type=HATE,inputStrength=HIGH,outputStrength=HIGH}]' \  
--sensitive-information-policy-config \  
'piiEntitiesConfig=[{type=EMAIL,action=ANONYMIZE},{type=SSN,action=BLOCK}]' \  
--topic-policy-config 'topicsConfig=[{name=Finance,definition=Investment advice,action=BLOCK}]' \  
# IAM policy – per-team model access \  
{ "Effect": "Allow", \  
  "Action": ["bedrock:InvokeModel", "bedrock:InvokeModelWithResponseStream"], \  
  "Resource": "arn:aws:bedrock*:::foundation-model/anthropic.claude-3-*", \  
  "Condition": { "StringEquals": { "aws:PrincipalTag/Team": "platform-eng" } } \  
}
```

VERDICT

Best choice for AWS-native organisations with compliance requirements where data must stay within AWS. Not suitable as a multi-cloud or multi-provider gateway. Pair with a provider-agnostic proxy for OpenAI/Azure traffic.

04

Azure API Management + AI Gateway

Microsoft's enterprise API platform extended for AI

Microsoft Azure (2024 AI extensions) · Proprietary (Azure managed) · Azure / M365 enterprise

Azure API Management (APIM) gains AI Gateway capabilities through the GenAI Gateway Accelerator and native Azure OpenAI integration. APIM's policy engine (XML-based) can enforce token limits, retry logic, semantic caching (via Azure Cache for Redis), and backend load balancing across multiple Azure OpenAI deployments. The integration with Azure AD, Managed Identity, and Microsoft Entra ID makes it the default choice for Microsoft-centric enterprises running Copilot, M365, and Azure OpenAI workloads.

Core Capabilities

Azure OpenAI Load Balancing	Token Rate Limiting	Semantic Cache (Redis)	Policy Engine (XML)	Managed Identity AuthN	Azure AD / Entra ID RBAC	Developer Portal	Mock Responses
Retry + Circuit Breaker	Cost Tracking (Azure Monitor)	Private Endpoints	Subscription Keys	Product-based Tenancy	OTel Export		

Capability Scores

Multi-provider routing		5/10
Extensibility / Plugins		6/10
Semantic caching		7/10
Security & guardrails		7/10
Observability		7/10
Ease of setup		7/10
Multi-tenant isolation		7/10
Multi-cloud support		3/10
OSS maturity		2/10
FinOps		7/10

Pros / Cons / Best Practices / Anti-Patterns

■ PROS	✗ CONS
• Native Azure OpenAI integration with deployment-level load balancing and PTU management • Managed Identity eliminates credential management — no API keys in code • Developer Portal provides self-service API discovery and subscription management • XML policy engine is powerful and well-documented for common gateway patterns • Product-based subscriptions map naturally to tenant/team quota management • Azure Monitor + App Insights provide integrated observability without extra tooling • Private Endpoint support keeps all traffic within Azure backbone • GenAI Gateway accelerator provides reference implementation for token tracking + retry	• XML policy language is verbose and difficult to test — no unit test framework for policies • Semantic cache is add-on requiring Azure Cache for Redis — not built in • Multi-cloud routing to non-Azure providers is possible but cumbersome • Performance overhead of APIM policy engine compared to Nginx-native gateways • Enterprise tier required for VNet injection and private endpoints (expensive) • No native prompt injection detection — requires Azure AI Content Safety as separate service • APIM v2 (2024) improves performance but the migration path from v1 is disruptive • Lock-in to Azure policy engine — migration to another gateway requires full rewrite
★ BEST PRACTICES	■ ANTI-PATTERNS
→ Use Managed Identity for Azure OpenAI auth — never use api-key header in production → Implement token rate limiting policy using the estimate-token-count + rate-limit-by-key built-in → Load balance across multiple Azure OpenAI deployments in different regions for PTU spillover → Use Named Values for all configuration parameters — never hardcode in policy XML → Enable Application Insights correlation header injection for distributed traces across Azure services → Create APIM Products per tenant/team — each product maps to a subscription with quota → Deploy APIM in internal mode within a VNet for zero-public-exposure architecture → Use the GenAI Gateway accelerator Bicep templates as baseline — do not start from scratch	! Writing complex business logic in APIM policy XML — use Azure Functions backends for logic > 20 lines ! Using Consumption tier for production AI workloads — no VNet support, cold starts on first request ! Hardcoding Azure OpenAI endpoint URLs in policy — use Named Values + Key Vault references ! Sharing one APIM subscription key across all teams — zero granularity for cost attribution ! Relying on APIM built-in cache for AI responses without Redis — limited to 5min TTL and 8KB response ! Not setting backend timeout policies — long LLM responses exceed default 30s gateway timeout ! Deploying without API revision management — breaking changes to AI routes cause immediate consumer impact

Reference Snippet

■ Azure APIM – Token rate limiting + multi-backend LB policy (snippet)

```
<policies>
<inbound>
```

```
<authentication-managed-identity resource='https://cognitiveservices.azure.com'/>
<!-- Token estimation & rate limiting -->
<azure-openai-token-limit
counter-key='@(context.Subscription.Id)'
tokens-per-minute='50000'
estimate-prompt-tokens='true'
remaining-tokens-header-name='x-ratelimit-remaining-tokens'/>
<!-- Semantic cache lookup -->
<azure-openai-semantic-cache-lookup
score-threshold='0.85'
embeddings-backend-id='ada-embedding-backend'/>
<!-- Load balance across 2 AOAI deployments -->
<set-backend-service backend-id='@(
context.Request.Headers["x-priority"] == "high"
? "aoai-swedencentral-ptu"
: "aoai-eastus-paygo")'/>
</inbound>
<outbound>
<azure-openai-semantic-cache-store duration='3600'/>
</outbound>
</policies>
```

VERDICT	Best choice for Azure-native enterprises standardised on Azure OpenAI and Microsoft identity. The Managed Identity + Entra ID integration is unmatched in the Microsoft ecosystem. Not recommended for multi-cloud or OpenAI-direct workloads.
---------	--

05	Portkey.ai	Developer-first AI gateway with prompt versioning	Portkey (2023) · SaaS / OSS (portkey-gateway on GitHub) · Dev teams / Scale-ups
----	------------	---	---

Portkey is a developer-first AI gateway and observability platform. Its OSS core (portkey-gateway) provides multi-provider routing, fallbacks, retries, and caching. The SaaS platform adds prompt management, version control, A/B testing, analytics, and feedback loops. Portkey's prompt vault enables versioned, templated prompts managed centrally — a capability absent from infrastructure-focused gateways. It is the leading choice for product teams who treat prompts as first-class engineering artifacts.

Core Capabilities

Multi-provider Routing	Provider Fallback	Semantic Cache	Retry Logic	Prompt Vault	Prompt Versioning & A/B Test	Request Tracing	Cost Analytics
Virtual Keys	Guardrails (SaaS)	Feedback Loops	SDKs (Python/JS/Go)		Metadata Tagging	Webhook Integrations	

Capability Scores

Multi-provider routing		9/10
Extensibility / Plugins		6/10
Semantic caching		7/10
Security & guardrails		6/10
Observability		9/10
Ease of setup		9/10
Multi-tenant isolation		6/10
Multi-cloud support		8/10
OSS maturity		6/10
FinOps		8/10

Pros / Cons / Best Practices / Anti-Patterns

■ PROS

- Prompt Vault with versioning, A/B testing, and rollback — unique among gateways
- Fastest time-to-value — SDK drop-in, production in hours not days
- Best-in-class request tracing with per-request cost breakdown and latency waterfall
- Virtual keys with per-key model allow-list, budget, and metadata tagging
- Feedback loop API allows apps to mark responses as helpful/harmful — feeds analytics
- OSS portkey-gateway can be self-hosted for data-residency requirements
- Comprehensive SDK for Python, JavaScript, and Go with OpenAI-compatible interface
- Guardrails on SaaS tier with PII detection and content moderation

✗ CONS

- SaaS-first — self-hosted OSS lacks the full analytics and prompt management features
- Multi-tenant isolation is virtual-key-scoped, not process/namespace isolated
- No native Kubernetes Operator — deployment requires manual Helm/Docker management
- Guardrails are less configurable than Kong plugins or AWS Bedrock Guardrails
- Vendor dependency for prompt vault — migrating prompts out of Portkey requires export tooling
- Limited network-level security controls compared to infrastructure gateways
- Enterprise SLAs and support less established than Kong or AWS offerings
- Concurrent request limits on lower SaaS tiers can be hit by high-volume teams

★ BEST PRACTICES

- Store all production prompts in Portkey Prompt Vault — never hardcode prompts in application code
- Use A/B testing for every significant prompt change before full rollout
- Tag every request with metadata (user_id, feature_id, session_id) for granular analytics
- Use virtual keys per microservice — rotate keys without changing application config
- Enable semantic cache with score_threshold=0.85 and monitor cache_hit_rate in dashboard
- Set up Portkey webhooks to fire on guardrail triggers — integrate with PagerDuty or Slack
- Self-host portkey-gateway on Kubernetes for HIPAA/GDPR workloads — disable SaaS telemetry
- Use Portkey feedback API to collect thumbs-up/down signals — feed into prompt optimisation loop

■ ANTI-PATTERNS

- ! Storing prompts in application code alongside Portkey SDK — defeats the entire prompt management value prop
- ! Using a single virtual key for all teams — lose granular cost attribution and cannot revoke per team
- ! Assuming SaaS Portkey is HIPAA-compliant by default — check BAA availability and data processing agreement
- ! Not versioning prompts before A/B tests — no rollback path when variant performs worse
- ! Relying on Portkey caching alone without TTL tuning — stale cached responses in dynamic-data contexts
- ! Using Portkey for security-critical guardrails without layering infrastructure controls — single point of bypass

Reference Snippet

■ Portkey – Multi-provider config with fallback + prompt vault (snippet)

```
from portkey_ai import Portkey
portkey = Portkey()
```

```
api_key='pk-virtual-key-team-a',
config={
  'strategy': { 'mode': 'fallback' },
  'targets': [
    { 'virtual_key': 'openai-prod', 'weight': 1 },
    { 'virtual_key': 'azure-oai', 'weight': 1 }, # fallback
    { 'virtual_key': 'anthropic-prod', 'weight': 1 }, # fallback
  ],
  'cache': { 'mode': 'semantic', 'max_age': 3600 },
  'retry': { 'attempts': 3, 'on_status_codes': [429, 503] },
}
)
# Use versioned prompt from Vault
resp = portkey.prompts.completions.create(
  prompt_id='prod-summariser-v3',
  variables={ 'document': doc_text },
)
```

VERDICT

Best choice for product teams who treat prompts as first-class engineering artifacts. The prompt vault and A/B testing are unique. For security-critical or multi-tenant enterprise deployments, layer with infrastructure-level controls.

06

Cloudflare AI Gateway

Edge-native AI gateway with global PoPs and WAF

Cloudflare (2024) · SaaS (Cloudflare managed) · Web-first / Global SaaS

Cloudflare AI Gateway runs at the edge of Cloudflare's global network (300+ PoPs), providing the lowest-latency first-hop for AI requests from globally distributed users. It combines AI-specific features (request logging, caching, rate limiting) with Cloudflare's existing WAF, DDoS protection, and Zero Trust (WARP/Access) capabilities. The zero-infrastructure model (workers-based, no servers to manage) makes it uniquely easy to deploy for teams already running on Cloudflare.

Core Capabilities

Global Edge Cache	Rate Limiting	Request Logging	WAF Integration	DDoS Protection	Zero Trust (Access)	Workers AI (on-device inference)	Provider Routing
Cost Analytics	Real-time Dashboard	Caching TTL Control	Retry Logic	Custom Workers Middleware	AI Binding (Workers AI)		

Capability Scores

Multi-provider routing		6/10
Extensibility / Plugins		5/10
Semantic caching		6/10
Security & guardrails		8/10
Observability		7/10
Ease of setup		10/10
Multi-tenant isolation		5/10
Multi-cloud support		6/10
OSS maturity		1/10
FinOps		6/10

Pros / Cons / Best Practices / Anti-Patterns

■ PROS

- Fastest setup of any gateway — operational in minutes via Cloudflare dashboard
- Global edge network eliminates origin latency for geographically distributed users
- Built-in WAF + DDoS protection on every AI request without additional configuration
- Workers AI provides on-device inference at the edge for low-latency, offline-capable apps
- Zero Trust integration gates AI access to authenticated corporate users (Cloudflare Access)
- Real-time request dashboard with latency, cost, and error metrics built in
- Caching automatically reduces costs for repeated requests — content-addressed

✗ CONS

- Semantic similarity caching is basic compared to vector-DB-backed solutions
- Multi-tenant isolation is limited — no workspace or namespace concept
- Extensibility requires Cloudflare Workers (JavaScript/WASM) — Lua/Python not supported
- Full prompt/response content logging raises GDPR concerns for EU data
- Vendor lock-in — all config lives in Cloudflare dashboard, no GitOps export
- No native OPA policy integration or structured RBAC beyond Access rules
- Provider support limited compared to LiteLLM (no on-prem models without Workers proxy)
- Pricing per request can become expensive at very high volume vs self-hosted alternatives

★ BEST PRACTICES

- Route only public-facing AI endpoints through Cloudflare AI Gateway — keep internal model traffic on private network
- Enable Cloudflare Access in front of AI Gateway for authenticated corporate AI tools
- Set cache TTL per endpoint based on content freshness requirements — never use default for dynamic data
- Use Workers middleware to inject tenant ID from Access JWT into gateway logs for attribution
- Enable rate limiting per Cloudflare Access user identity, not per IP — handles mobile/CGNAT
- Monitor Cloudflare Analytics for cache hit ratio — target > 30% for cost efficiency
- Use Cloudflare AI Gateway with Workers AI for edge inference to reduce round-trip to cloud providers

■ ANTI-PATTERNS

- ! Sending PHI/PII through Cloudflare AI Gateway SaaS logging without a signed BAA — HIPAA violation
- ! Assuming WAF rules cover prompt injection — they do not without custom WAF rules for LLM attack patterns
- ! Using Cloudflare AI Gateway as the only gateway layer for complex enterprise routing — it lacks RBAC depth
- ! Not disabling content logging for sensitive endpoints — all prompts/completions are stored in Cloudflare
- ! Treating Cloudflare cache as a semantic cache — it is exact-match content hash, not similarity-based
- ! Over-relying on Cloudflare for resilience — a Cloudflare-wide incident affects all AI endpoints simultaneously

Reference Snippet

■ Cloudflare AI Gateway – Workers middleware with tenant tagging (snippet)

```
// Cloudflare Worker wrapping AI Gateway
export default {
  async fetch(request, env) {
    // Extract tenant from Cloudflare Access JWT
    const identity = await env.ACCESS.getIdentity(request)
```

```
const tenantId = identity?.custom?.tenant_id ?? 'unknown'
// Forward to Cloudflare AI Gateway with metadata
const gwUrl = 'https://gateway.ai.cloudflare.com/v1/{account}/{gateway}'
const response = await fetch(`${gwUrl}/openai/chat/completions`, {
  method: 'POST',
  headers: {
    ...request.headers,
    'cf-ai-metadata': JSON.stringify({ tenant: tenantId }),
    'Authorization': `Bearer ${env.OPENAI_KEY}`,
  },
  body: request.body,
})
return response
}
```

VERDICT

Best choice for teams already on Cloudflare who need rapid deployment and global edge presence. Not suitable as the sole enterprise AI gateway — lacks RBAC depth and multi-tenant isolation. Excellent as a front-edge layer composited with a deeper gateway.

07

Google Apigee AI Gateway

Enterprise API management extended for Vertex AI and Gemini

Google Cloud / Apigee (2024) · Proprietary (GCP managed) · GCP enterprise

Apigee AI Gateway extends Google's enterprise API management platform with AI-specific capabilities: Vertex AI model routing, Gemini integration, token-aware rate limiting, semantic caching via Vertex AI Embeddings, and AI safety via Vertex AI content filtering. Apigee's proven policy engine (XML/JavaScript), developer portal, and analytics platform bring enterprise API governance to AI traffic. The deep GCP integration — Vertex AI, Gemini, Cloud Armor, Duet AI — makes it the natural fit for GCP-first organisations.

Core Capabilities

Vertex AI Model Routing	Gemini Native Integration	Token Rate Limiting	Semantic Cache (Vertex Embeddings)	Content Safety (Vertex AI)	Cloud Armor WAF	Identity Platform AuthN	API Products & Monetisation
Developer Portal	Analytics (Advanced API Ops)	Shared Flow Reuse	Private Cloud (Hybrid)	Cloud Trace Integration	gRPC Support		

Capability Scores

Multi-provider routing		5/10
Extensibility / Plugins		7/10
Semantic caching		8/10
Security & guardrails		7/10
Observability		8/10
Ease of setup		6/10
Multi-tenant isolation		8/10
Multi-cloud support		3/10
OSS maturity		1/10
FinOps		7/10

Pros / Cons / Best Practices / Anti-Patterns

■ PROS	✗ CONS
• Native Vertex AI and Gemini integration — zero adapter code for GCP model catalog • Shared Flows allow reusable AI policy bundles (auth, rate limit, cache) across all APIs • Developer Portal provides self-service AI model access with subscription and quota management • API Products and monetisation enable external AI product revenue with metered billing • Advanced API Operations provides ML-based anomaly detection on API traffic • Hybrid deployment (Apigee hybrid) allows on-prem data plane with GCP control plane • Cloud Armor integration provides DDoS and WAF protection without additional config	• Steep learning curve — Apigee policy XML + Shared Flows require dedicated expertise • Expensive — Apigee X pricing is among the highest in the API management market • Non-GCP provider routing (OpenAI, Anthropic direct) requires custom JavaScript policy • Semantic cache depends on Vertex Embeddings — additional cost per embedding call • No native prompt injection detection — requires Vertex AI Safety Attributes parsing in policy • Migration from Apigee Edge (legacy) to Apigee X is a major re-implementation project • Community support is limited; documentation quality varies across features
★ BEST PRACTICES	■ ANTI-PATTERNS
→ Use Shared Flows for all cross-cutting AI concerns — auth, token limit, cache, safety → Model Vertex AI backends as Apigee TargetServers — decouple endpoint URLs from policy → Use API Products to gate model access per team/product — each product maps to a Gemini model tier → Enable Advanced API Ops anomaly detection to catch AI-specific traffic anomalies → Apply Cloud Armor security policy at load balancer level before Apigee — first line of defence → Use Apigee hybrid data plane in EU/regulated regions with GCP control plane for data residency → Instrument all AI proxy flows with custom analytics variables (token counts, model, cost) for chargeback	! Writing AI routing logic directly in proxy flows instead of Shared Flows — creates policy duplication across hundreds of proxies ! Using Apigee Eval (free tier) for production AI workloads — no SLA, rate limited, not suitable for prod ! Not setting target timeout extensions for LLM streaming — default 55s Apigee timeout drops long completions ! Hardcoding Vertex AI project/location in policy — use environment-specific KVM (Key Value Map) instead ! Sharing one API Product across all consumers — no granular quota attribution ! Ignoring Apigee's quota sync latency (~30s) — brief over-quota requests possible in distributed deployments

Reference Snippet

■ Apigee AI Gateway – Vertex AI proxy with token rate limit + cache (snippet)

```
<!-- Apigee proxy flow – AI inbound policies -->
<Flow name='AI-Chat-Flow'>
  <Request>
    <!-- Auth via Google Identity Platform -->
    <Step><Name>VerifyIDToken</Name></Step>
    <!-- Token rate limiting from KVM -->
    <Step><Name>TokenRateLimitPolicy</Name></Step>
    <!-- Semantic cache lookup -->
```

```
<Step><Name>SemanticCacheLookup</Name></Step>
<!-- Route to Vertex AI Gemini -->
<Step><Name>SetVertexAITarget</Name></Step>
</Request>
<Response>
<!-- Store response in semantic cache -->
<Step><Name>SemanticCacheStore</Name></Step>
<!-- Extract token counts for analytics -->
<Step><Name>ExtractTokenUsage</Name></Step>
</Response>
</Flow>
```

VERDICT	Best choice for GCP-first enterprises with existing Apigee investment and Vertex AI/Gemini workloads. The Shared Flows and Developer Portal are powerful for large API programs. High cost and GCP lock-in make it unsuitable as a multi-cloud solution.
---------	--

08

WSO2 Choreo / API Manager

Open-source enterprise API gateway with AI extensions

WSO2 (2024 AI extensions) · Apache 2 (OSS) / Enterprise · OSS enterprise

WSO2 API Manager and its cloud-native evolution (Choreo) extend the mature open-source API management platform with AI-specific capabilities: LLM rate limiting, semantic caching, AI traffic analytics, and integration with OpenAI, Azure OpenAI, and Anthropic. WSO2's strength is its fully open-source, vendor-neutral positioning — no proprietary lock-in. Choreo adds a developer platform layer with built-in CI/CD, observability, and a component marketplace. Ideal for regulated industries that require source-code-visible infrastructure.

Core Capabilities

Multi-provider LLM Routing	Token Rate Limiting	AI Traffic Analytics	Semantic Cache	AI Subscription Management	Ballerina Integration	GraphQL AI APIs	Developer Portal
API Monetisation	Open-source Core	On-prem Deployment	Kubernetes Microgateway	Policy-as-Code	OIDC / OAuth2		

Capability Scores

Multi-provider routing		7/10
Extensibility / Plugins		7/10
Semantic caching		6/10
Security & guardrails		6/10
Observability		7/10
Ease of setup		5/10
Multi-tenant isolation		7/10
Multi-cloud support		7/10
OSS maturity		8/10
FinOps		6/10

Pros / Cons / Best Practices / Anti-Patterns

■ PROS	✗ CONS
• Fully open-source under Apache 2 — source-auditable for regulated industries • No vendor lock-in — deploy on any cloud, on-prem, or hybrid • Mature API management platform with 10+ years of production deployments • Kubernetes-native microgateway (Choreo Connect) for lightweight sidecar deployments • Ballerina language integrations provide type-safe AI service compositions • Built-in API marketplace and developer portal for internal AI product catalogue • Strong community in banking, healthcare, and government sectors • Policy-as-code approach aligns with enterprise governance frameworks	• AI-specific features are newer and less mature than Kong or LiteLLM equivalents • Complex installation — WSO2 API Manager has significant operational overhead • Smaller AI/ML community compared to LiteLLM or Portkey • Semantic cache implementation requires additional configuration versus out-of-box solutions • Limited prompt guard / injection detection — requires custom mediators • Documentation for AI features lags behind product releases • Choreo SaaS is regionally limited — self-hosted is the only option for all-region coverage
★ BEST PRACTICES	■ ANTI-PATTERNS
→ Use Choreo Connect (microgateway) for AI traffic — lighter weight than full API Manager for LLM routes → Define AI APIs as separate API Products in WSO2 Developer Portal with distinct subscription tiers → Implement token counting in custom mediation sequences before enforcing rate limits → Use WSO2 Key Manager with external identity providers (Keycloak, Azure AD) for federated authN → Enable API Analytics for AI traffic — monitor latency, error rates, and consumer usage trends → Run WSO2 API Manager on Kubernetes with Operator — enables GitOps-managed API lifecycle → Use Ballerina AI library for type-safe multi-provider orchestration in integration flows	! Deploying full WSO2 API Manager for a simple LLM proxy — use Choreo Connect microgateway instead ! Using WSO2's default in-memory cache for AI responses — switch to Redis for production semantic cache ! Not separating the API Manager control plane from Choreo Connect data plane — single failure domain ! Treating WSO2 subscriptions as security boundaries alone — enforce network policies separately ! Using XML-based synapse mediators for new AI integrations — prefer Ballerina or REST connector approach

Reference Snippet

■ WSO2 Choreo Connect – AI API rate limiting policy (snippet)

```
# Choreo Connect – AI API definition with token rate limit
openapi: '3.0.0'
info:
  title: AI Chat API
x-wso2-throttling-tier: AIGoldTier
x-wso2-cors:
```

```
corsConfigurationEnabled: true
paths:
  /chat/completions:
    post:
      x-auth-type: Application
      x-throttling-tier: AITokenLimit_100K_PerMin
      x-wso2-backend:
        url: https://api.openai.com/v1/chat/completions
        type: HTTP
      security:
        - apiKey: []
```

VERDICT	Best choice for regulated industries (banking, healthcare, government) requiring fully open-source, source-auditable AI gateway infrastructure. Expect higher operational effort than managed alternatives. Not yet AI-feature-complete compared to Kong or LiteLLM.
---------	--

09

Traefik AI Plugin

Kubernetes-native lightweight AI proxy via plugin

Traefik Labs (2024) · Apache 2 (Traefik OSS) / Enterprise plugin · Cloud-native / DevOps teams

Traefik Proxy's plugin ecosystem enables AI gateway capabilities through middleware plugins. The Traefik AI plugin suite provides LLM request routing, rate limiting, and observability as Traefik middlewares, making it a natural fit for Kubernetes-native teams already using Traefik as their ingress controller. The sidecar-compatible model allows AI middleware to sit directly in the request path without introducing a separate gateway process. Traefik Hub (enterprise) adds API management capabilities including rate limiting, analytics, and RBAC.

Core Capabilities

Middleware Plugin Architecture	K8s IngressRoute CRDs	LLM Request Routing	Rate Limiting	Circuit Breaker	Retry Middleware	OpenTelemetry	Prometheus Metrics
mTLS	Let's Encrypt TLS	Header Manipulation	Load Balancing	Traefik Hub RBAC	Sidecar Deployment Model		

Capability Scores

Multi-provider routing		6/10
Extensibility / Plugins		7/10
Semantic caching		3/10
Security & guardrails		5/10
Observability		7/10
Ease of setup		8/10
Multi-tenant isolation		5/10
Multi-cloud support		6/10
OSS maturity		8/10
FinOps		4/10

Pros / Cons / Best Practices / Anti-Patterns

■ PROS	✗ CONS
• Native Kubernetes CRD-first design — IngressRoute, Middleware as K8s objects • Zero additional infrastructure for teams already running Traefik as ingress controller • Lightweight — Traefik process handles both general API and AI traffic in one binary • Excellent dynamic configuration — routes update without restarts via K8s watch • Strong open-source community and extensive plugin marketplace • mTLS and Let's Encrypt automation out of the box • Sidecar deployment enables per-service AI middleware without central gateway bottleneck	• AI-specific features (semantic cache, prompt guard, PII detection) require custom plugins • No built-in semantic cache — requires external Redis + custom plugin development • No native virtual key or budget management for multi-tenant AI workloads • Traefik Hub required for enterprise RBAC and analytics — free tier is limited • Plugin development requires Go knowledge — steeper curve than Lua or Python • Cost tracking and FinOps capabilities require external tooling integration • Not designed as a purpose-built AI gateway — AI features are additive, not core
★ BEST PRACTICES	■ ANTI-PATTERNS
→ Use IngressRoute CRDs with Middleware chain for all AI routes — keep config declarative and GitOps-managed → Deploy Traefik with circuit breaker middleware targeting LLM provider backends (networkError threshold: 5) → Use Traefik's retry middleware with exponential backoff for 429 and 503 responses → Enable OpenTelemetry tracing with baggage propagation for distributed traces through AI service chains → Implement rate limiting middleware using Redis-backed distributed token bucket for multi-replica accuracy → Use Traefik Hub API Portal for developer-facing AI API catalogue with subscription management → Pair Traefik with a dedicated LiteLLM sidecar for provider abstraction — Traefik handles ingress, LiteLLM handles LLM routing	! Using Traefik as the sole AI gateway without a provider abstraction layer — creates provider-specific route proliferation ! Relying on in-memory rate limiting with multiple replicas — counters are per-replica, not shared ! Building complex AI logic (prompt guard, PII detection) as Traefik plugins — use sidecar service instead ! Not setting circuit breaker response conditions for LLM-specific errors (non-2xx from provider) ! Using Traefik's default timeout for LLM streaming routes — default 30s drops long completions

Reference Snippet

■ Traefik – AI route with rate limit + circuit breaker middleware (snippet)

```
# Traefik IngressRoute + Middleware (Kubernetes CRD)
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata: { name: ai-rate-limit }
spec:
  rateLimit:
    average: 100
```

```
burst: 20
period: 1m
sourceCriterion:
  requestHeaderName: X-Tenant-ID
---
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata: { name: ai-circuit-breaker }
spec:
  circuitBreaker:
    expression: ResponseCodeRatio(500,600,0,600) > 0.30
    checkPeriod: 10s
    fallbackDuration: 30s
---
apiVersion: traefik.io/v1alpha1
kind: IngressRoute
metadata: { name: ai-gateway-route }
spec:
  routes:
  - match: PathPrefix('/ai/v1')
  middlewares:
  - name: ai-rate-limit
  - name: ai-circuit-breaker
  services:
  - name: litellm-svc
port: 4000
```

VERDICT	Best choice for Kubernetes-native teams running Traefik who want AI middleware without introducing a new gateway process. Pair with LiteLLM for provider abstraction. Not a standalone enterprise AI gateway — purpose-built AI gateways (Kong, LiteLLM) should be preferred for complex AI governance needs.
---------	---

10	IBM watsonx Gateway	Enterprise AI governance gateway for regulated industries	IBM (2024) · Proprietary (IBM Cloud / on-prem) · Financial / Government / Healthcare
----	---------------------	---	--

IBM watsonx.ai and the associated API Connect AI Gateway provide enterprise AI governance capabilities targeting heavily regulated industries. The platform combines IBM's API Connect gateway engine with watsonx model catalog access, AI Factsheets (model governance records), OpenScale bias detection, and Watson Trust capabilities. IBM's on-premises deployment model and air-gapped operation support make it the default for financial institutions and government agencies that cannot use public cloud AI services.

Core Capabilities

watsonx Model Catalog	AI Factsheets (Model Cards)	Bias & Drift Detection	API Connect Gateway	On-prem / Air-gapped Deployment	IBM Security Verify AuthN	Z/OS Integration	FIPS 140-2 Encryption
DataStage Integration	Watson NLU Guardrails	IBM Cloud IAM	Audit Trail (Immutable)	SLA Management	Private Model Hosting		

Capability Scores

Multi-provider routing		4/10
Extensibility / Plugins		5/10
Semantic caching		4/10
Security & guardrails		9/10
Observability		8/10
Ease of setup		3/10
Multi-tenant isolation		8/10
Multi-cloud support		4/10
OSS maturity		2/10
FinOps		6/10

Pros / Cons / Best Practices / Anti-Patterns

■ PROS

- Industry-leading AI governance — AI Factsheets document every model's lineage, bias metrics, and drift
- On-prem and air-gapped deployment for data that cannot leave corporate infrastructure
- FIPS 140-2 encryption compliance for financial and government workloads
- Deep integration with IBM mainframe (Z/OS) and DataStage for legacy data pipelines
- Watson NLU-based content safety with domain-specific financial/legal classifiers
- IBM Security Verify provides enterprise-grade identity and privileged access management
- Immutable audit trail with chain-of-custody for regulatory examination
- IBM enterprise support with contractual SLAs suitable for Tier-1 banking workloads

✗ CONS

- Highest total cost of ownership of any gateway in this comparison
- Extremely steep learning curve — requires IBM-certified architects for implementation
- Primarily optimised for IBM model ecosystem — other providers require complex adapters
- Slow product innovation cycle compared to cloud-native alternatives
- Semantic caching and developer experience lag behind modern gateways significantly
- Not suitable for rapid prototyping or startup use cases
- Multi-cloud portability is limited — primarily IBM Cloud or on-prem
- Community ecosystem is minimal — IBM partner/professional services dependent

★ BEST PRACTICES

- Create AI Factsheets for every model deployed — this is the core governance value of the platform
- Use IBM Security Verify Privileged Access Management for all watsonx admin credentials
- Deploy on IBM Cloud Pak for Data for the most integrated on-prem watsonx experience
- Leverage DataStage integration to govern training data pipelines alongside inference governance
- Enable OpenScale continuous bias monitoring — set alert thresholds based on regulatory risk appetite
- Use API Connect products and plans to create tiered access to AI models per internal team
- Run air-gapped deployments on IBM Z for maximum data sovereignty in financial institutions

■ ANTI-PATTERNS

- ! Selecting IBM watsonx Gateway for use cases that do not require on-prem or regulated deployment — significant over-engineering
- ! Treating AI Factsheets as a one-time setup exercise — they must be updated on every model retrain or version change
- ! Using IBM watsonx for rapid experimentation cycles — the procurement and setup timeline is months, not hours
- ! Not engaging IBM Professional Services for initial deployment — DIY implementations frequently misconfigure governance modules
- ! Ignoring OpenScale drift detection alerts — the value is in acting on drift, not just measuring it

Reference Snippet

■ IBM watsonx – AI Factsheet metadata registration (concept snippet)

```
# IBM AI Factsheet registration via Python SDK (snippet)
from ibm_watson_openscale import APIClient
from ibm_watson_openscale.supporting_classes.enums import *
client = APIClient(authenticator=authenticator)
client.ai_factsheets.create_model_entry(
```

```
model_id='watsonx-granite-13b-v2',
metadata={
  'model_type': 'foundation_model',
  'framework': 'IBM Granite',
  'training_data_ref': 's3://corp-data/training/v2',
  'bias_metrics': { 'demographic_parity': 0.02 },
  'data_residency': 'US',
  'regulatory_scope': ['FINRA', 'SOX'],
  'approved_by': 'AI Ethics Committee 2026-03-01',
}
```

VERDICT	Only justified for heavily regulated industries (Tier-1 banking, defence, government) with on-prem mandates and existing IBM infrastructure. The AI governance capabilities are unmatched for these use cases. For all other scenarios, modern cloud-native alternatives deliver better ROI.
---------	--

HOW TO CHOOSE: Decision Guide

Match your primary driver to the recommended gateway. Most enterprises will layer 2–3 solutions.

I need maximum extensibility & plugin customisation	Kong AI Gateway
I need broadest provider coverage, Python-native	LiteLLM Proxy
I am all-in on AWS and need zero infra management	AWS Bedrock Gateway
I am all-in on Azure / Microsoft 365 / Copilot	Azure APIM AI Gateway
I need prompt versioning & A/B testing as a priority	Portkey.ai
I need edge delivery + WAF + zero infra setup	Cloudflare AI Gateway
I am all-in on GCP / Vertex AI / Gemini	Google Apigee AI Gateway
I need fully open-source, vendor-neutral, regulated sector	WSO2 Choreo / API Manager
I run Traefik on Kubernetes and want AI middleware	Traefik AI Plugin
I need on-prem, air-gapped, AI governance + IBM ecosystem	IBM watsonx Gateway
Multi-cloud: agnostic routing across all providers	LiteLLM + Kong (layered)
Multi-tenant SaaS with hard isolation per customer	Kong (Silo/Bridge) + Redis
Edge + enterprise: global users + deep governance	Cloudflare (edge) + Kong (origin)

This comparison reflects product capabilities as of April 2026. AI gateway features evolve rapidly — validate scores against current vendor documentation before architectural commitment. Scores are the authors' assessment of out-of-box capability; heavy customisation can raise scores in any dimension.